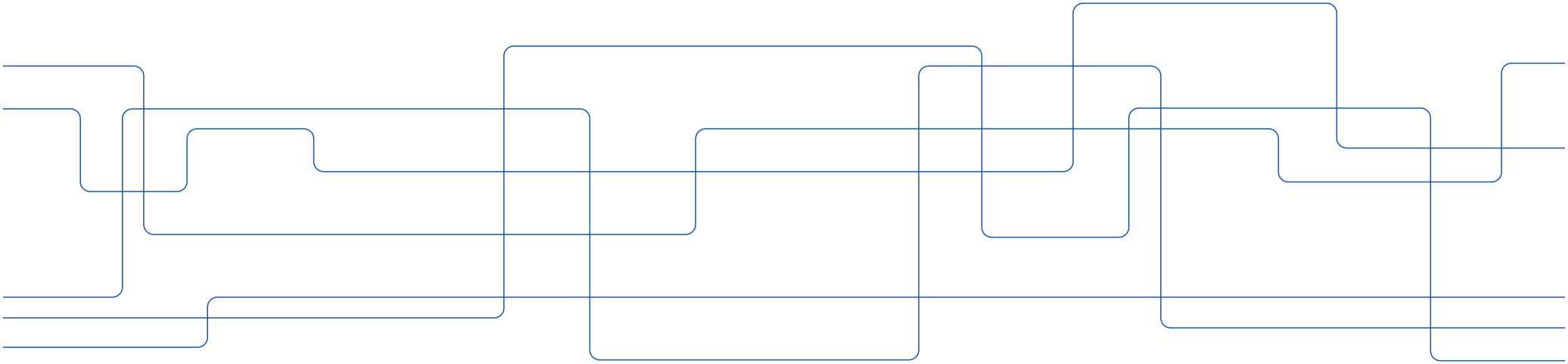


HI1024

Grundkurs i programmering

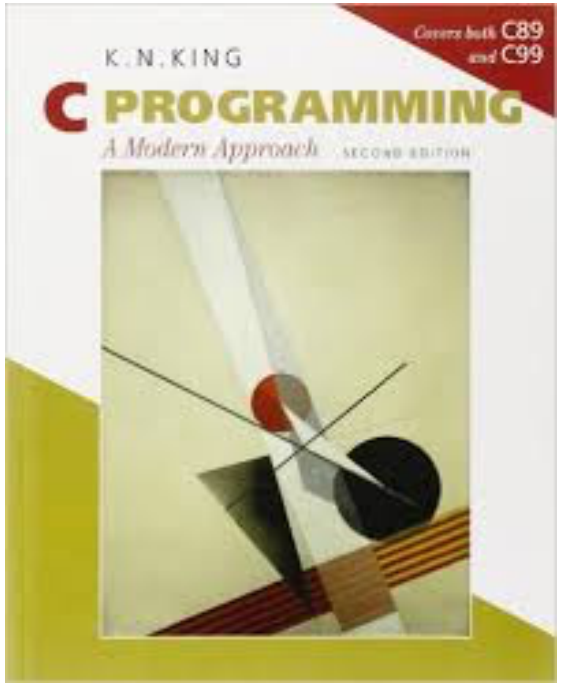
F8: Funktioner – på djupet!

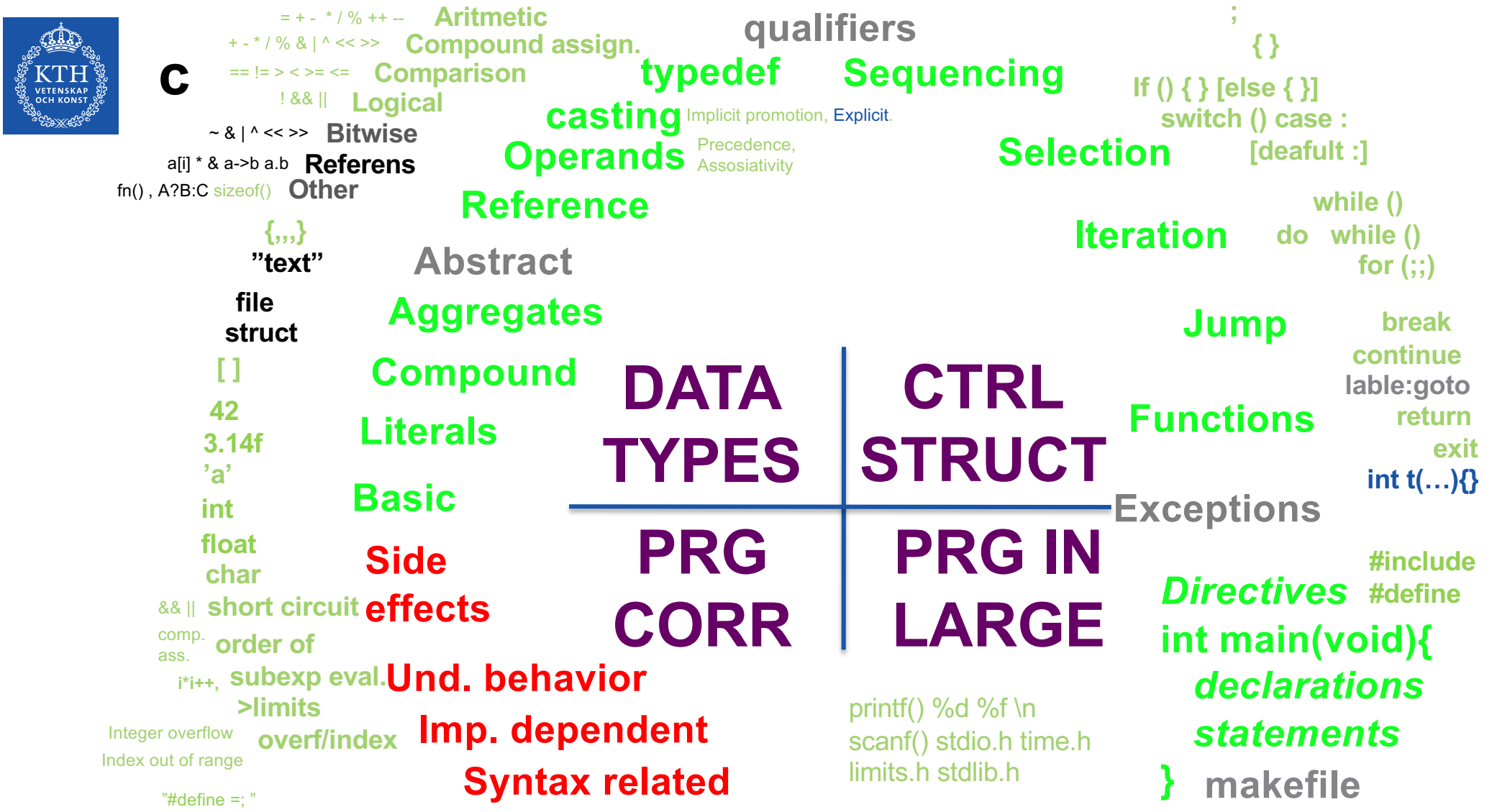


Innehåll!

King 9.6 & 10

- Repetition föreläsning #6
- Rekursion
- Lokala vs globala variabler
- Programutvecklingsmetodik
- Övning




C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Literals
Compound
Aggregates
Abstract
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence,
Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
Side effects
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
Und. behavior
>limits
Integer overflow
Index out of range
overf/index
Imp. dependent
”#define =; ”
Syntax related
printf() %d %f \n
scanf() stdio.h time.h
limits.h stdlib.h
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
int/int
/*comment*/ or //

Funktioner



```
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
        printf("Dice %d : %d\n", i+1, (rand() % SIZE) + 1);

    printf("MT Done.\n");
    return 0;
}
```

Måste (i detta fall) stå före (första) anropet!

Ingen ut-data! **Namn** **Ingen in-data!**

```
void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}
```

Benämns: Funktionsdefinition

Lättbegripligt!

~~Obegripligt!~~

Återanvändbart.

Ingen in-data!

Ingen ut-data!

Funktioner

Ett typiskt c-program innehåller 100-tals, för att inte säga 1000-tals funktioner…

…för att inte main() ska hamna sist så är **Best Practice** att berätta för kompilatorn att funktionen finns genom en så kallad funktionsdeklaration som är funktionsdefinitionen fram till ’{’ som ersätts med ett ’;’…

…informationen är nog för att kompilatorn ska kunna göra sitt arbete, i förvissningen om att en fullständig funktionsdefinition kommer innan filen är slut!

Funktionsdefinition →

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, (rand() % SIZE) + 1);

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}
```

← **Funktionsdeklaration**

Vi vill att kod som ni redovisar ser ut på detta sätt. Under utveckling är det ok att börja med enbart definitionen.

Funktioner

Funktionsdeklaration

(Kan flyttas till en separate .h-fil)

Funktionsanrop

Funktionsdefinition

(Kan flyttas till en separate .c-fil)

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(void);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice());

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(void) {
    return (rand() % SIZE) + 1;
}
```

Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(int size);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice(SIZE));

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(int size) {

    return (rand() % size) + 1;
}
```



```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE    NAMN        TYP
#F000:    0        i         int
#F001:    6      size        int
#F002:    ?        ?          ?
#F003:
#F004:
#F005:
#F006:
```

R

…ny variabel size som ska vara 6…

Observera att size är en ny (lokal) (parameter-)variabel, som får en egen plats i minnet!



Nytt: Funktioner



Med andra ord: En funktion ska om möjligt vara ett fåtal antal satser som utför någon ofta återkommande, specifik, uppgift! Behovet av en funktion identifieras lättas:

- Under arbetet med en funktion (i vårt fall main()) noterar vi att det finns:
    1. Komplicerad/Obegriplig kod
    2. Återanvändbar kod
    3. Funktionen håller på att bli för stor (mer än en skärmsida).

- Under ”skissarbetet”, innan vi skriver någon kod, inser vi att det finns en handfull funktioner som skulle göra allt mycket enklare – ofta kopplat till de datastrukturer programmet kommer att använda sig av!



Nytt: Funktioner



En funktion utvecklas på samma sätt som när vi har skapat huvudfunktionen main()!

- Behöver funktionen någon speciell datastruktur/algoritm (0)
- Skissa en lösning till funktionens kod! (1)
- Skapa enbart anropet, och testa att det fungerar! (2)
- Utveckla funktionen, steg för steg, testa! (3..4)
- (Spara testprogrammet som testar funktionen!)
- Använd funktionen!

Observera: Den första koden som skrivs är testprogrammet som kan anropa den blivande funktionen!

1-dimentionella

Matriser som parametrar

```
#include <stdio.h>
#define WEEKDAYS 7

                        1       2
void printBox(int aBox[], int size){
   for (int i=0; i<size; i++)
      printf("Box[%d] : %d\n", i, aBox[i]);
}

int main(int argc, char **argv) {
   int box[WEEKDAYS]={2,1,1,1,0,0,1};
   printf("Matrix Tester (MT)!\n");
            3
   printBox(box, WEEKDAYS);

   printf("MT Done, bye!\n");
   return 0;
}
```

```
Matrix Tester (MT)!
Box[0] : 2
Box[1] : 1
Box[2] : 1
Box[3] : 1
Box[4] : 0
Box[5] : 0
Box[6] : 1
MT Done, bye!
```

4

R

```
 PROGRAM
#0000: CODE
   *
 DATA  VÄRDE      NAMN              TYP
#F000:   2    box[0] aBox[0]       int
#F001:   1    box[1] aBox[1]       int
#F002:   1    box[2] aBox[2]       int
#F003:   1    box[3] aBox[3]       int
#F004:   0    box[4] aBox[4]       int
#F005:   0    box[5] aBox[5]       int
#F006:   1    box[6] aBox[6]       int
#F007:   7    size                 int
#F008:   0    i                    int
#F009:
```

1: Parameter saknar storlek!
2: Storleken bifogas separat!
3: Argumentet saknar []
4: Den (till synes) nya var. aBox tilldelas **samma plats i minnet** som argumentet!!!
(Helt tvärt emot hur en int parameter skapades)

!!! Skriver printBox() i variabel aBox kommer det efteråt att märkas i variabel box, används mycket ofta!!!



2-dimentionella

Matriser som parametrar

Tic tac toe

```c
int main(int argc, char **argv){
    char b[S][S];
    char r='0',c='0'; int turn=0;

    printf("Tic Tac Toe!\n");
    printf("Enter rowcol\n");
    printf("or 00 to end\n");
    initBoard(b, S);
    do {
        printBoard(b, S);

        printf("Where?");
        scanf(" %c%c",&r,&c);

        if (r<'1' || c<'1')
            turn=0;
        else {
            if (++turn%2)
                b[r-'1'][c-'1']='X';
            else
                b[r-'1'][c-'1']='0';
        }
    } while (turn);
    printf("Done, bye!\n");
    return 0;
}
```



```c
#include <stdio.h>
#define S 3

void printBoard(char aB[][S], int rows){
    for (int row=0; row<rows; row++) {
        for (int col=0; col<S; col++)
            printf("%c", aB[row][col]);
        printf("\n");
    }
}
```

1 2

```c
void initBoard(char aB[][S], int rows){
    for (int row=0; row<rows; row++) {
        for (int col=0; col<S; col++)
            aB[row][col]='+';
    }
}
```

R

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE     NAMN          TYP
#F000:   ?   b[0][0]ab[0][0] char
#F001:   ?   b[0][1]ab[0][1] char
#F002:   ?   b[0][2]ab[0][2] char
#F003:   ?   b[1][0]ab[1][0] char
#F004:   ?   b[1][1]ab[1][1] char
#F005:   ?   b[1][2]ab[1][2] char
#F006:   ?   b[2][0]ab[2][0] char
#F007:   ?   b[2][1]ab[2][1] char
#F008:   ?   b[2][2]ab[2][2] char
#F009: '0'   r               char
#F00A: '0'   c               char
#F00B:   0   turn            int
#F00C:   3   rows            int
```

VIKTIGT!

1) 1:a dimensionen enbart []
2) 2:a dimensionen storlek [3]

Observera hur matrisparam. hamnar på samma ställe som argumentet, medan int param. får en ny minnescell!

Reflektioner övning #7

```
Monte Carlo 1.0
[Spade  K] 10 h(it) or s(top) ? h
[Heart  Q] 20 h(it) or s(top) ? s
[Heart  J] 10 h(it) or s(top) ? h
[Spade  9] 19 h(it) or s(top) ? h
[Heart  9] 28 BUSTED!
*** WINN :-)
Program completed, bye!
```

Saknas!

```c
int main(int argc, char **argv) {
    char cmd='s';
    int card;
    int player=0;

    printf("Monte Carlo 0.95\n");
    initRandomGenerator();

    do {
        card = rndNumber(DECK);
        player=player+getRank(card);
        printCard(card);
        printf("%2d ", player);
        if (player<BUSTED) {
            printf("h(it) or s(top) ? ");
            scanf(" %c", &cmd);
        } else {
            printf("BUSTED!\n");
            cmd='s';
        }
    } while (cmd=='h');

    printf("Program completed, bye!\n");
    return 0;
}
```

Hum…
…det saknas kod här…
…datorn ska ”spela”…
…minst 17 för att stanna!

Reflektioner övning #7

```
Monte Carlo 1.0
[Spade  K] 10 h(it) or s(top) ? h
[Heart  Q] 20 h(it) or s(top) ? s
[Heart  J] 10 h(it) or s(top) ? h
[Spade  9] 19 h(it) or s(top) ? h
[Heart  9] 28 BUSTED!
*** WINN :-)
Program completed, bye!
```

Saknas!

BRA!

```c
int main(int argc, char **argv) {
    char cmd='s';
    int card;
    int player=0;
    int cpu=0;
    printf("Monte Carlo 0.95\n");
    initRandomGenerator();

    do {
        card = rndNumber(DECK);
        player=player+getRank(card);
        printCard(card);
        printf("%2d ", player);
        if (player<BUSTED) {
            printf("h(it) or s(top) ? ");
            scanf(" %c", &cmd);
        } else {
            printf("BUSTED!\n");
            cmd='s';
        }
    } while (cmd=='h');

    printf("Program completed, bye!\n");
    return 0;
}
```

```c
if (player<BUSTED) {
    do {
        card = rndNumber(DECK);
        cpu = cpu + getRank(card);
        printCard(card);
        printf("%2d ", cpu);
        if(cpu < BUSTED) {
            printf("h(it) or s(top) ? ");
            if((cpu > 16) && cpu >= player)
                cmd = 's';
            else
                cmd = 'h';
            printf("%c\n", cmd);
        } else {
            printf("BUSTED!\n");
            cmd = 's';
        }
    } while (cmd == 'h');
}

if (cpu==player)
    printf("+++ DRAW :-!\n");
else
    if ( ((cpu>player) && (cpu<BUSTED)) || !cpu )
        printf("*** LOSE :-(\n");
    else
        printf("*** WINN :-)\n");
```

Hum… …i stort samma kod?!

Reflektioner övning #7

```
Monte Carlo 1.0
[Spade  K] 10 h(it) or s(top) ? h
[Heart  Q] 20 h(it) or s(top) ? s
[Heart  J] 10 h(it) or s(top) ? h
[Spade  9] 19 h(it) or s(top) ? h
[Heart  9] 28 BUSTED!
*** WINN :-)
Program completed, bye!
```

Saknas!

(I ett kommersiellt program hade alla ”1:or” ersatts med t.ex USR och #define USR 1 samt alla 0:or med CPU och #define CPU 0)

```c
int main(int argc, char **argv) {
    char cmd='s';
    int card;
    int player[2]={0,0};

    printf("Monte Carlo 1.1\n");
    initRandomGenerator();

    for (int p=1; p>=0 && player[1]<BUSTED; p--)
        do {
            card = rndNumber(DECK);
            player[p] = player[p] + getRank(card);
            printCard(card);
            printf("%2d ", player[p]);
            if(player[p] < BUSTED) {
                printf("h(it) or s(top) ? ");
                if (p)
                   scanf(" %c", &cmd);
                else {
                   if((player[0] > 16) &&
                      (player[0] >= player[1]) )
                      cmd = 's';
                   else
                      cmd = 'h';
                   printf("%c\n", cmd);
                }
            } else {
                printf("BUSTED!\n");
                cmd = 's';
            }
        } while (cmd == 'h');

    if (player[0]==player[1])
        printf("+++ DRAW :-!\n");
    else
        if ( ((player[0]>player[1]) &&
              (player[0]<BUSTED)) || !player[0] )
            printf("*** LOSE :-(\n");
        else
            printf("*** WINN :-)\n");

    printf("Program completed, bye!\n");
    return 0;
}
```

Varför göra 2 olika int, när vi kan göra en int matris, och använda index i stället!

BÄTTRE!

Är kodpartier snarlika, placera raderna i en loop eller gör en funktion av dem!

(Nu är det också enkelt att utöka till 2,3..n spelare och en bank, ett tecken på bra kod!)

Mjukstart…

```c
#include <stdio.h>

int main(){

   // Code

   printf("Available commands:\n");
   printf("1 = View\n");
   printf("2 = Edit\n");
   printf("3 = Save\n");
   printf("4 = Cancel\n");

   // Code

   printf("Available commands:\n");
   printf("1 = View\n");
   printf("2 = Edit\n");
   printf("3 = Save\n");
   printf("4 = Cancel\n");

   // Code

   printf("Available commands:\n");
   printf("1 = View\n");
   printf("2 = Edit\n");
   printf("3 = Save\n");
   printf("4 = Cancel\n");

}
```

Mjukstart…

Returnerar inget!

Kräver inget!

```c
#include <stdio.h>

int main(){

   // Code

   printf("Available commands:\n");
   printf("1 = View\n");
   printf("2 = Edit\n");
   printf("3 = Save\n");
   printf("4 = Cancel\n");

   // Code

   printf("Available commands:\n");
   printf("1 = View\n");
   printf("2 = Edit\n");
   printf("3 = Save\n");
   printf("4 = Cancel\n");

   // Code

   printf("Available commands:\n");
   printf("1 = View\n");
   printf("2 = Edit\n");
   printf("3 = Save\n");
   printf("4 = Cancel\n");

}
```

```c
#include <stdi

void printMenu(void);

int main(){

   // Code

   printMenu();

   // Code

   printMenu();

   // Code

   printMenu();
}

void printMenu(void) {
   printf("Available commands:\n");
   printf("1 = View\n");
   printf("2 = Edit\n");
   printf("3 = Save\n");
   printf("4 = Cancel\n");
}
```

Indata? Utdata?

Funktionsdeklaration/definition!

Mjukstart…

```c
#include <stdio.h>

int main(){
    char ch;
    int ok;

   // Code

   printf("Are you really sure [Y/N] ? ");
   scanf(" %c", &ch);
   ok = ((ch=='Y') || (ch=='y'));
   if (ok)
      ; //do something

   // Code

   printf("Are you really sure [Y/N] ? ");
   scanf(" %c", &ch);
   ok = ((ch=='Y') || (ch=='y'));
   if (ok)
      ; //do something

   // Code

   printf("Are you really sure [Y/N] ? ");
   scanf(" %c", &ch);
   ok = ((ch=='Y') || (ch=='y'));
   if (ok)
      ; //do something
}
```

Mjukstart…

```c
#include <stdio.h>

int main(){
    char ch;
    int ok;

   // Code

   printf("Are you really sure [Y/N] ? ");
   scanf(" %c", &ch);
   ok = ((ch=='Y') || (ch=='y'));
   if (ok)
      ; //do something

   // Code

   printf("Are you really sure [Y/N] ? ");
   scanf(" %c", &ch);
   ok = ((ch=='Y') || (ch=='y'));
   if (ok)
      ; //do something

   // Code

   printf("Are you really sure [Y/N] ? ");
   scanf(" %c", &ch);
   ok = ((ch=='Y') || (ch=='y'));
   if (ok)
      ; //do something
}
```

Returnerar int!

Kräver inget!

```c
#include <stdi

int reallySure(void);

int main(){

   // Code

   if (reallySure())
      ; //do something

   // Code

   if (reallySure())
      ; //do something

   // Code

   if (reallySure())
      ; //do something
}

int reallySure(void){
   char ch;

   printf("Are you really sure [Y/N] ?");
   scanf(" %c", &ch);
   return ((ch=='Y') || (ch=='y'));
}
```

Lokal variabel

Indata? Utdata?

Minst ett (1) return!

Mjukstart…

```c
#include <stdio.h>

int main(){
    int r,a,b,c,d,e,f;

   // Code

   if (a>b)
      r=a;
   else
      r=b;

   // Code

   if (c>d)
      r=c;
   else
      r=d;

   // Code

   if (e>f)
      r=e;
   else
      r=f;
}
```

Mjukstart…

```c
#include <stdio.h>

int main(){
    int r,a,b,c,d,e,f;

   // Code

   if (a>b)
      r=a;
   else
      r=b;

   // Code

   if (c>d)
      r=c;
   else
      r=d;

   // Code

   if (e>f)
      r=e;
   else
      r=f;
}
```

Returnerar int! Kräver 2 int!

```c
int max(int x, int y);

int main(){
    int r,a,b,c,d,e,f;

   // Code

   r=max(a,b);

   // Code

   r=max(c,d);

   // Code

   r=max(e,f);
}

int max(int x, int y){
   if(x>y)
     return x;
   else
     return y;
}
```

Argumenten och parametrarna är olika variabler även om de haft samma namn!

x blir inte e, utan x är en ny variabel som får en kopia av e:s värde! (copy-by-value)

Hade funktionen ändrat på x:s värde hade det inte påverkat e:s värde!

Indata? Utdata?

Minst ett (1) return!

Mjukstart…

```c
#include <stdio.h>
#define SIZE 8

int main(){
    char theBoard[SIZE][SIZE];

   // Code

   for (int r=0; r<SIZE; r++)
      for (int c=0; c<SIZE; c++)
          theBoard[r][c]='.';

   // Code

   for (int r=0; r<SIZE; r++)
      for (int c=0; c<SIZE; c++)
          theBoard[r][c]='+';

   // Code

   for (int r=0; r<SIZE; r++)
      for (int c=0; c<SIZE; c++)
          theBoard[r][c]='*';
}
```

Mjukstart…

```c
#include <stdio.h>
#define SIZE 8

int main(){
    char theBoard[SIZE][SIZE];

   // Code

   for (int r=0; r<SIZE; r++)
      for (int c=0; c<SIZE; c++)
          theBoard[r][c]='.';

   // Code

   for (int r=0; r<SIZE; r++)
      for (int c=0; c<SIZE; c++)
          theBoard[r][c]='+';

   // Code

   for (int r=0; r<SIZE; r++)
      for (int c=0; c<SIZE; c++)
          theBoard[r][c]='*';
}
```

Returnerar inget!

Kräver 2-dim mat, 1 int & 1char!

```c
#include <stdio.h>
#define SIZE 8

void initBoard(char aBoard[][SIZE], int rows, char character);

int main(){
    char theBoard[SIZE][SIZE];

   // Code

   initBoard(theBoard, SIZE, '.');

   // Code

   initBoard(theBoard, SIZE, '+');

   // Code

   initBoard(theBoard, SIZE, '*');
}

void initBoard(char aBoard[][SIZE], int rows, char character){
   for (int r=0; r<rows; r++)
      for (int c=0; c<SIZE; c++)
          aBoard[r][c]=character;
}
```


Indata? Utdata?

Obs: theBoard & aBoard ligger på samma ställe i minnet!

Nytt: Fälla!

```
#include <stdio.h>
#define WEEKDAYS 7

int main(int argc, char **argv) {
    int box[WEEKDAYS]={2,1,1,1,0,0,1};
    printf("Matrix tester (MT)!\n");

    for (int i=0; i<WEEKDAYS; i++)
        printf("Box[%d] : %d\n", i, box[i]);

    printf("MT Done, bye!\n");
    return 0;
}
```

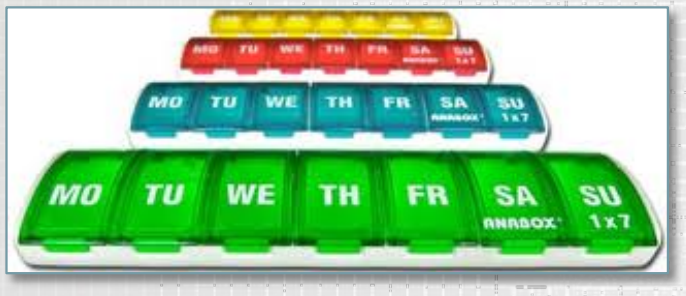
Kan skrivas som!

7 / 1 = 7

```
for (int i=0; i<sizeof(box)/sizeof(box[0]); i++)
    printf("Box[%d] : %d\n", i, box[i]);
```

```
Matrix Tester (MT)!
Box[0] : 2
Box[1] : 1
Box[2] : 1
Box[3] : 1
Box[4] : 0
Box[5] : 0
Box[6] : 1
MT Done, bye!
```

```
 PROGRAM
#0000: CODE
   *
 DATA  VÄRDE      NAMN          TYP
#F000:   2    box[0]           int
#F001:   1    box[1]           int
#F002:   1    box[2]           int
#F003:   1    box[3]           int
#F004:   0    box[4]           int
#F005:   0    box[5]           int
#F006:   1    box[6]           int
#F007:
#F008:
#F009:
```



Tanke: Dela storleken på matrisen med storleken på ett element för att få antalet element, kan fungera men…

Nytt: Fälla!

```c
#include <stdio.h>
#define WEEKDAYS 7

void printBox(int aBox[]) {
   for (int i=0; i<sizeof(aBox)/sizeof(aBox[0]); i++)
       printf("Box[%d] : %d\n", i, aBox[i]);
}

int main(int argc, char **argv) {
    int box[WEEKDAYS]={2,1,1,1,0,0,1};
    printf("Matrix tester (MT)!\n");

    printBox(box);

    printf("MT Done, bye!\n");
    return 0;
}
```

2 / 1 = 2

```
Matrix tester (MT)!
Box[0] : 2
Box[1] : 1
MT Done, bye!
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE      NAMN          TYP
#F000:   2    box[0]aBox[0]    int
#F001:   1    box[1]aBox[1]    int
#F002:   1    box[2]aBox[2]    int
#F003:   1    box[3]aBox[3]    int
#F004:   0    box[4]aBox[4]    int
#F005:   0    box[5]aBox[5]    int
#F006:   1    box[6]aBox[6]    int
#F007:
#F008:
#F009:
```

…en matris parameter är i själva verket enbart adressen till första elementet, därför behöves en parameter till (eller ett annat trick) för att tala om hur många element som faktiskt finns…

…i den här dator är en adress tydligen dubbelt så minneskrävande som en int!

…använda aldrig sizeof() för att beräkna hur många element som finns (om du inte är otroligt säker på din sak)!

Nytt: Rekursion

En funktion kan anropa sig själv, det sägs då vara ett rekursivt anrop!

T.ex. kan fakulteten av ett tal n lätt beräknas som:
n! = n * (n – 1)! Där n! = 1.

```
int fact(int n){
  if (n<=1) 4
    return 1;
  else                  3
    return n * fact(n-1);
}
```

```
printf("4!=%d",fact(4));
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE      NAMN          TYP
#F000:  4         n            int
#F001:
#F002:
#F003:
#F004:
#F005:
#F006:
#F007:
#F008:
#F009:
```

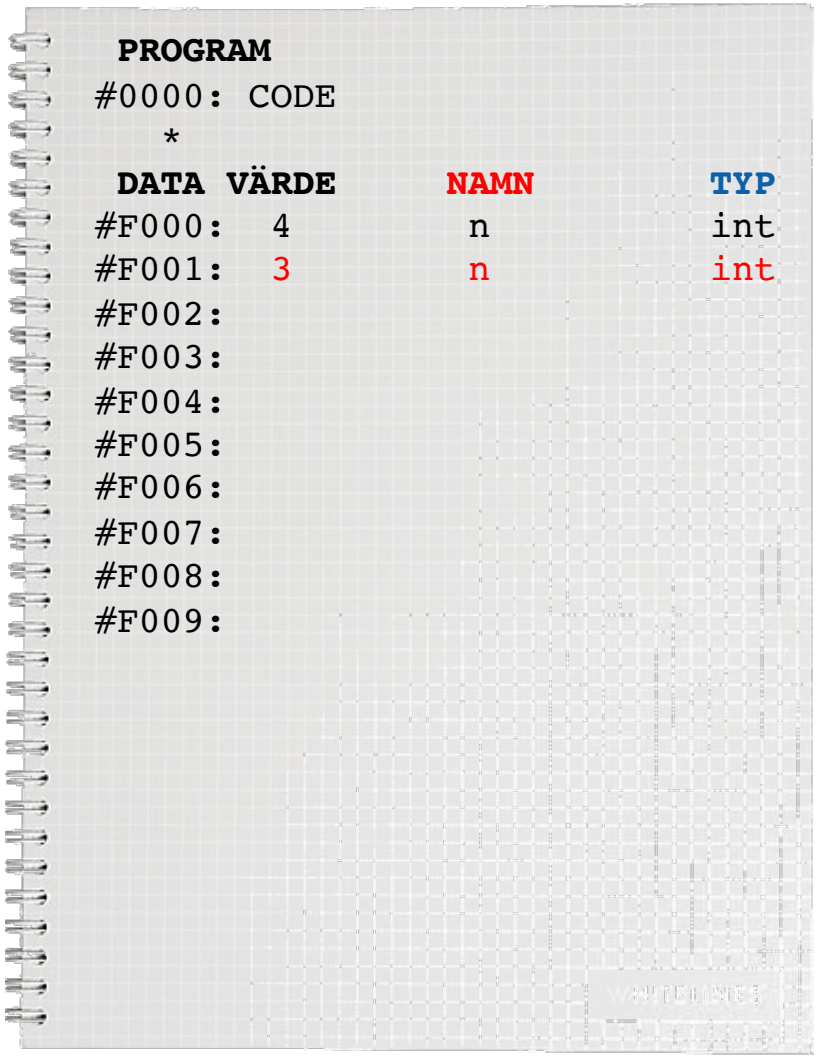
Nytt: Rekursion

En funktion kan anropa sig själv, det sägs då vara ett rekursivt anrop!

T.ex. kan fakulteten av ett tal n lätt beräknas som:
n! = n * (n – 1)! Där n! = 1.

```
int fact(int n){
  int fact(int n){
    if (n<=1) 3
      return 1;
    else                    2
}     return n * fact(n-1);
  }
```

```
printf("4!=%d",fact(4));
```



Nytt: Rekursion

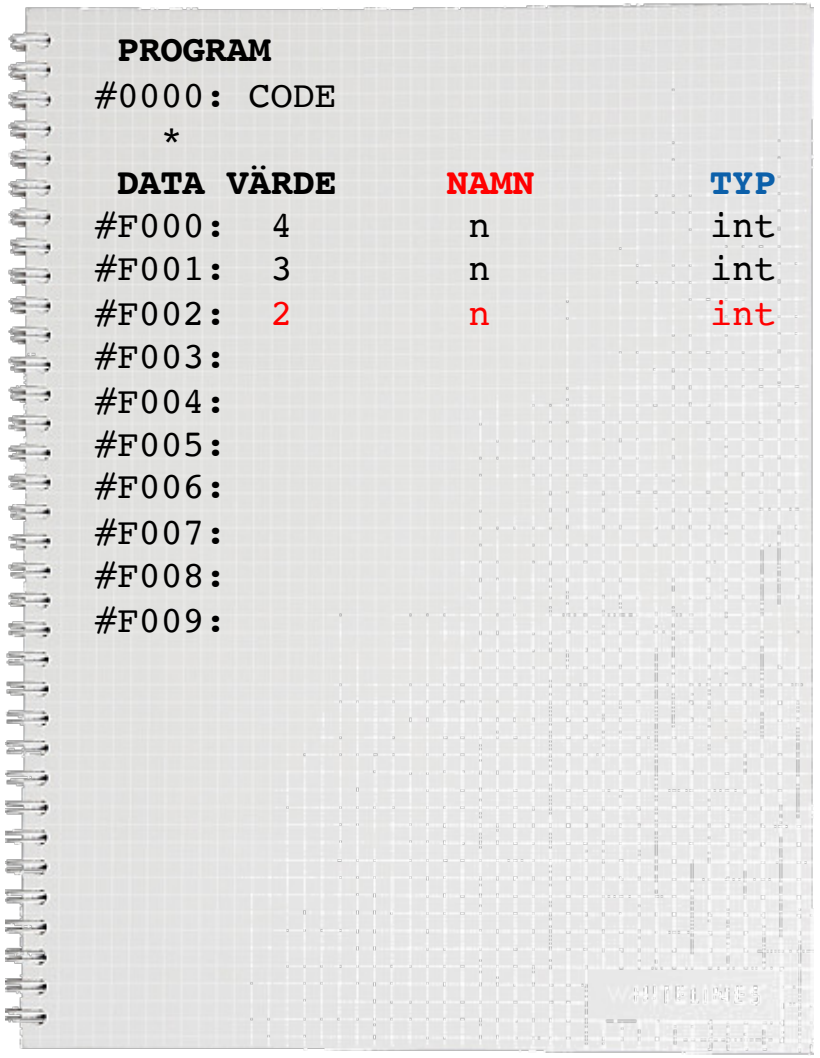
En funktion kan anropa sig själv, det sägs då vara ett rekursivt anrop!

T.ex. kan fakulteten av ett tal n lätt beräknas som:
n! = n * (n – 1)! Där n! = 1.

```
int fact(int n){
  int fact(int n){
    int fact(int n){
      if (n<=1) 2
        return 1;
}     else
  }     return n * fact(n-1);
    }
```

1

```
printf("4!=%d",fact(4));
```

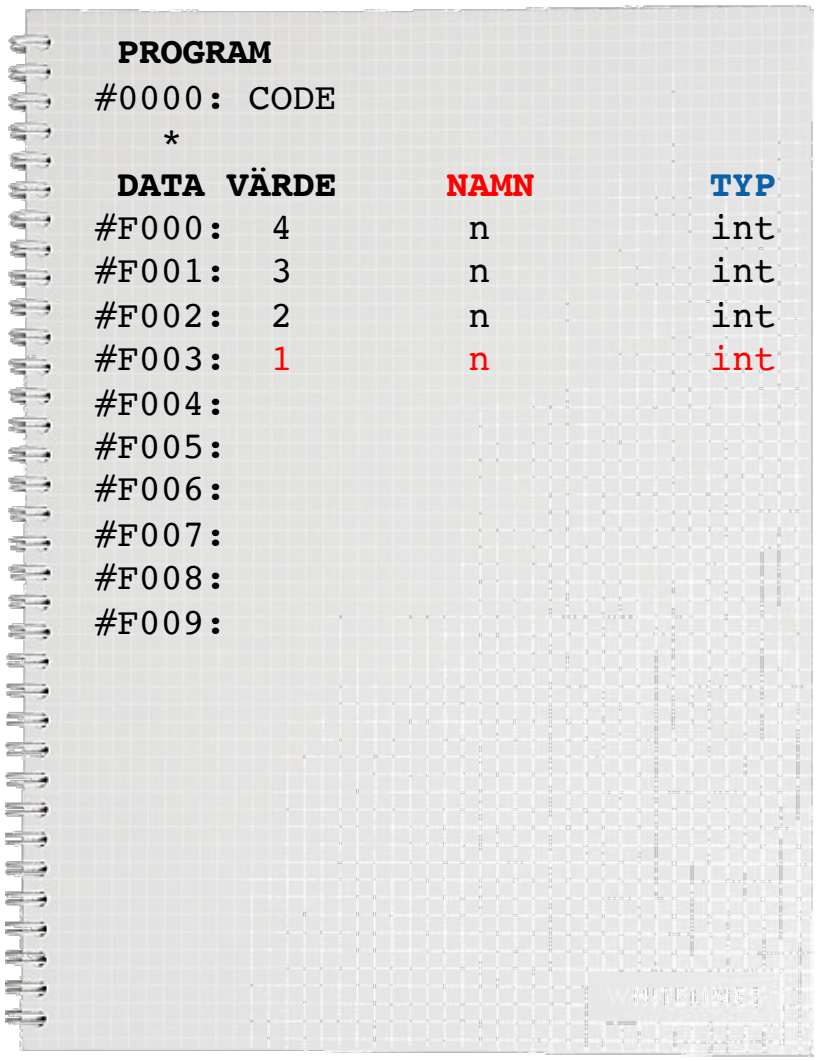


Nytt: Rekursion

En funktion kan anropa sig själv, det sägs då vara ett rekursivt anrop!

T.ex. kan fakulteten av ett tal n lätt beräknas som:
n! = n * (n – 1)! Där n! = 1.

```
int fact(int n){
  int fact(int n){
    int fact(int n){
      int fact(int n){
        if (n<=1)
          return 1;
      }
      else
        return n * fact(n-1);
    }
  }
printf("4!=%d",fact(4));
```



Nytt: Rekursion

En funktion kan anropa sig själv, det sägs då vara ett rekursivt anrop!

T.ex. kan fakulteten av ett tal n lätt beräknas som:
n! = n * (n – 1)! Där n! = 1.

```
int fact(int n){
  int fact(int n){
    int fact(int n){
      if (n<=1)
        return 1;
    }   else
        return n * fact(n-1);
  }
}
```

2 * 1 = 2

```
printf("4!=%d",fact(4));
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE      NAMN          TYP
#F000:  4         n            int
#F001:  3         n            int
#F002:  2         n            int
#F003:
#F004:
#F005:
#F006:
#F007:
#F008:
#F009:
```

Nytt: Rekursion

En funktion kan anropa sig själv, det sägs då vara ett rekursivt anrop!

T.ex. kan fakulteten av ett tal n lätt beräknas som:
n! = n * (n – 1)! Där n! = 1.

```
int fact(int n){
  int fact(int n){
    if (n<=1)
      return 1;
    else
      return n * fact(n-1);
}   }
```

3 * 2 = 6

```
printf("4!=%d",fact(4));
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE      NAMN          TYP
#F000:  4         n            int
#F001:  3         n            int
#F002:
#F003:
#F004:
#F005:
#F006:
#F007:
#F008:
#F009:
```

Nytt: Rekursion

En funktion kan anropa sig själv, det sägs då vara ett rekursivt anrop!

T.ex. kan fakulteten av ett tal n lätt beräknas som:
n! = n * (n – 1)! Där n! = 1.

```
int fact(int n){
  if (n<=1)
    return 1;
  else
    return n * fact(n-1);
}
```

4 * 6 = 24

```
printf("4!=%d",fact(4));
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE      NAMN          TYP
#F000:  4         n            int
#F001:
#F002:
#F003:
#F004:
#F005:
#F006:
#F007:
#F008:
#F009:
```

Nytt: Rekursion

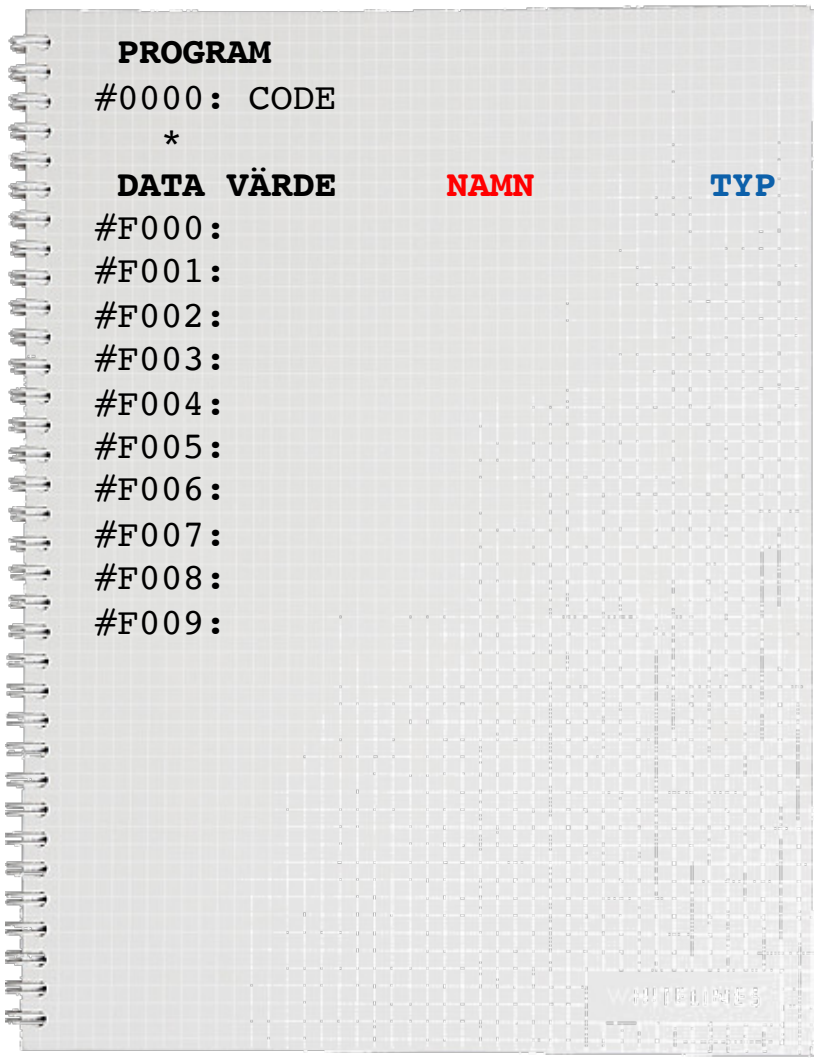
En funktion kan anropa sig själv, det sägs då vara ett rekursivt anrop!

T.ex. kan fakulteten av ett tal n lätt beräknas som:
n! = n * (n – 1)! Där n! = 1.

```
int fact(int n){
  if (n<=1)
    return 1;
  else
    return n * fact(n-1);
}
```

```
printf("4!=%d",fact(4));
```

24



Lokala vs Globala variabler

```c
/* Character 0.4 AC */
#include <stdio.h>
#define WEEKDAYS 7
#define DAYSLOTS 4

void printBox(int theBox[][WEEKDAYS], int rows);

int main(void){

    int w[DAYSLOTS][WEEKDAYS]={ {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7} };

    printf("Matrix Tester (MT)!\n");

    printBox( w, DAYSLOTS );

    printf("MT Done.\n");
    return 0;
}

void printBox(int theBox[][WEEKDAYS], int rows) {
   for (int i=0; i<rows; i++)
        for (int j=0; j<WEEKDAYS; j++)
      printf("Box[%d,%d] : %d\n",i,j,theBox[i][j]);
}
```

Variabeln w sägs vara lokal därför att den deklarerats inuti en funktion!

En lokal variabel skapas varje gång omslutande funktion anropas under exekvering, och den glöms bort så snart funktionen når sitt slut!

M.a.o kan den hamna på olika ställen i minnet från gång till gång…

Automatic storage duration, Block scope (visability)

En lokal variabel ”syns” inte utanför sin funktion, värdet kan sändas som argument till en annan funktion!

Lokala vs Globala variabler

Variabeln w sägs vara global därför att den deklarerats utanför alla funk!

En global variabel skapas när koden passeras under exekvering där den är deklarerad, och den finns kvar så länge kod från samma fil exekveras.

En global variabel är synlig i hela filen från den punkt där den deklareras! Den kan dock skuggas av lokala variabler/parametrar.

Static storage duration, File scope (visability)

```c
/* Character 0.4 AC */
#include <stdio.h>
#define WEEKDAYS 7
#define DAYSLOTS 4

void printBox();

    int w[DAYSLOTS][WEEKDAYS]={ {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7} };

int main(void){

    printf("Matrix Tester (MT)!\n");

    printBox( );

    printf("MT Done.\n");
    return 0;
}

void printBox() {
   for (int i=0; i<DAYSLOTS; i++)
        for (int j=0; j<WEEKDAYS; j++)
      printf("Box[%d,%d] : %d\n",i,j,w[i][j]);
}
```


Om en global variabel fått ett felaktigt värde är det mycket svårt att ta reda på vart, vi använder det inte i kursen!



Programutvecklingsmetodik

Den enkla metodik vi använt så här långt fungerar utmärkt om det är ett begränsat problem som vi ska skriva kod för, ett litet program eller en modul till ett större program…

…är problem svårare måste det först brytas ner till hanterbara mindre problem som vi nu kan lösa…

…hur det går till är stoff för flera års studier med det är definitivt en iterativ process där tanken är att nerlagt arbete successivt ska ge större och större insikt i hur problemet bör tacklas!

Algoritmer & Datastrukturer **0**

1 Skriv ner proble-met så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! **2**

3 a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

Bottom up vs Top Down



Du har fått i uppgift att skapa programvaran för en ny generation tvättmaskiner!

Bottom up vs Top Down



Wow!!!! den måste
ha en brutal motor!

Bottom up vs Top Down

Sen behövs det kanske någon form av styrning också...



Wow!!!! den måste ha en brutal motor!

Som de knyter samman till större funktionella enheter!

Bottom up vs Top Down

Javisst, en på-knapp behövs också…

Sen behövs det kanske någon form av styrning också...



Wow!!!! den måste ha en brutal motor!

Bottom up vs Top Down

Den behöver ett bra gränssnitt…



Andra föredrar att börja med ett tomt skal som kan ge en antydan om vad det ska bli!

Bottom up vs Top Down



Den behöver ett bra gränssnitt…

…som exponerar styrdatorns alla program…

Bottom up vs Top Down



Den behöver ett bra gränssnitt…

…som exponerar styrdatorns alla program…

…så att motorn körs på rätt sätt!

Bottom up vs Top Down

Top Down, givet ett grundlig arbete med system arkitekturen, brukar vara bäst!

Javisst, en på-knapp behövs också…

Sen behövs det kanske någon form av styrning också...

Wow!!!! den måste ha en brutal motor!



Den behöver ett bra gränssnitt…

…som exponerar styrdatorns alla program…

…så att motorn körs på rätt sätt!

Top Down ger också något som går att testa/visa upp för satsningens intressenter tidigt!

I undantagsfall går det att bygga fast sig i en felaktig sys.ark, men då har någon fuskat när satsningen började!



Top Down

- Titta på problemet från ett användarperspektiv
- Skapa en systemarkitektur genom att dela upp problemet i allt mindre byggblock tills varje byggblock är överblickbart, om möjligt också frikopplat från andra byggblock (få beroenden) samt potentiellt återanvändbart!
- Implementera användargränssnittet, både för att möjliggöra enkel testning av underliggande byggblock allt eftersom de blir klara men även för att kunna ge en bild av vad som byggs för satsningen intressenter.
- Ett byggblock benämns ofta modul (eng: unit) och när systemarkitekturarbetet är klart bör det finnas en specifikation på huvuddelen/alla moduler som behövs inklusive exakt vilka funktioner de ska erbjuda (eng: interface specification).
- Med lite tur så finns kanske flertalet av modulerna redan då de utvecklats för andra satsningar, de som återstår måste utvecklas, och vissa kan kanske återanvändas senare.

Varje enskild modul kan därefter utvecklas enligt 0..4 metoden!



Best Practic

- En driven programmerar har i bakhuvudet vilka moduler som redan finns under systemarkitekturarbetet, kan en befintlig modul användas kan det spara både tid och pengar.
- **En modul ska om möjligt inte avslöja något om hur den skapar erbjudna tjänster, utan bara vilka det är, detta kallas abstraktion!**
- I kursen har vi redan skapat 2 återanvändbara moduler med 2 tjänster var:

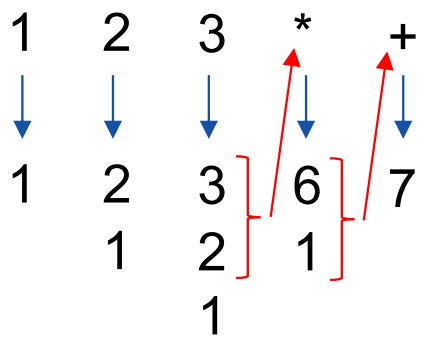
initRandomGenerator()
Återanvändbara slumpfunktioner!
rndNumber(52)

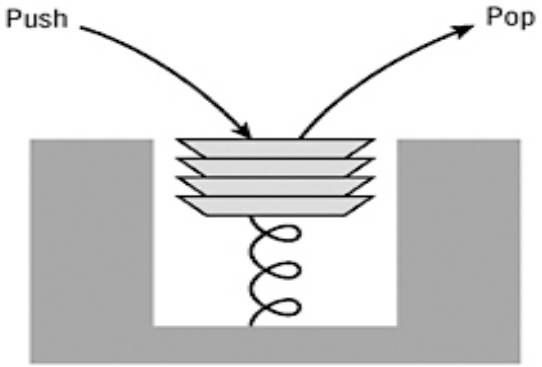
printCard(theCard)
Återanvändbara kortfunktioner!
getRank(theCard)

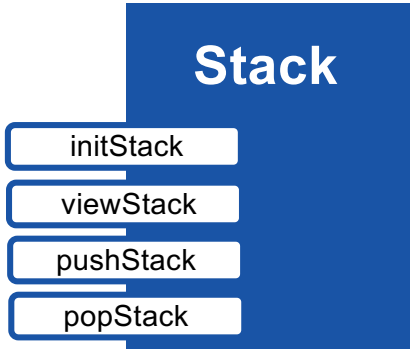
- Att samla på sig testade och beprövade moduler är nyckeln till framgång i denna bransch! Dags att träna på att skapa en till mycket användbar modul!

RPN miniräknare, Chapter10: P6

- Företaget du arbetar för tänker lansera ett litet miniräknarprogram där tal matas in med hjälp av den så kallade omvänd polsk notation (HP). Sättet är ursmart då det inte kräver parenteser eller lika med (=), se exempel…
- Arbetssättet för att lösa: 1 2 3 * + (= 7) är som följer:
- Tänk dig att du har en fjädrande behållare som ”talrikshållaren i matsalen”. Tal ”sparas”, om det däremot är en operation så tas de två översta talen från behållaren och operationen utförs innan svaret sparas i behållaren! M.a.o

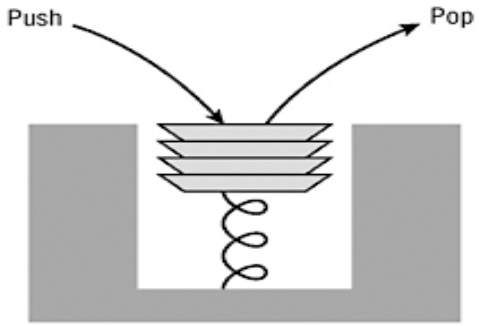


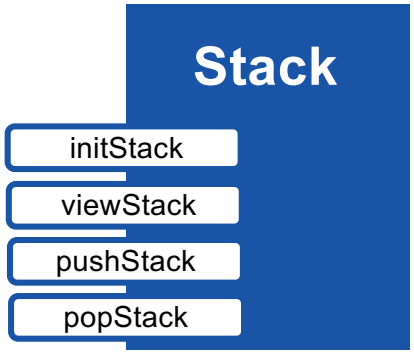




”Tallrikshållaren” kallas för en stack, tänk om vi haft en sådan…

RPN miniräknare, Chapter10: P6





int float

Vilken datatyp hjälper oss mest?!

char []

Algoritmer & Datastrukturer **0**

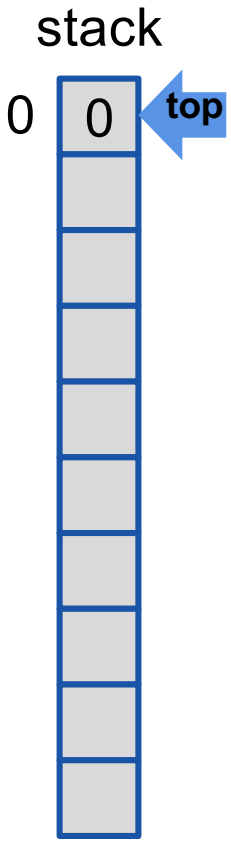
1 Skriv ner problemet så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! **2**

3 a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

RPN miniräknare, Chapter10: P6



Hum…

…en 1-dim matris kan kanske användas…

…det är svårt att ”trycka ner” tal…

…men ingen vet om de hamnar ”sist” i stället…

…indexet top får markera antalet element på stacken…

…**push** ökar top med ett, och placerar elementet på den utpekade positionen…

…**pop** returnerar sist lagrade element och räknar ner top med ett…

... varför inte spara indexet top först i matrisen*…

…ok att skippa alla tester för fel.

Algoritmer & Datastrukturer **0**

1 Skriv ner problemet så att ett barn förstår det!

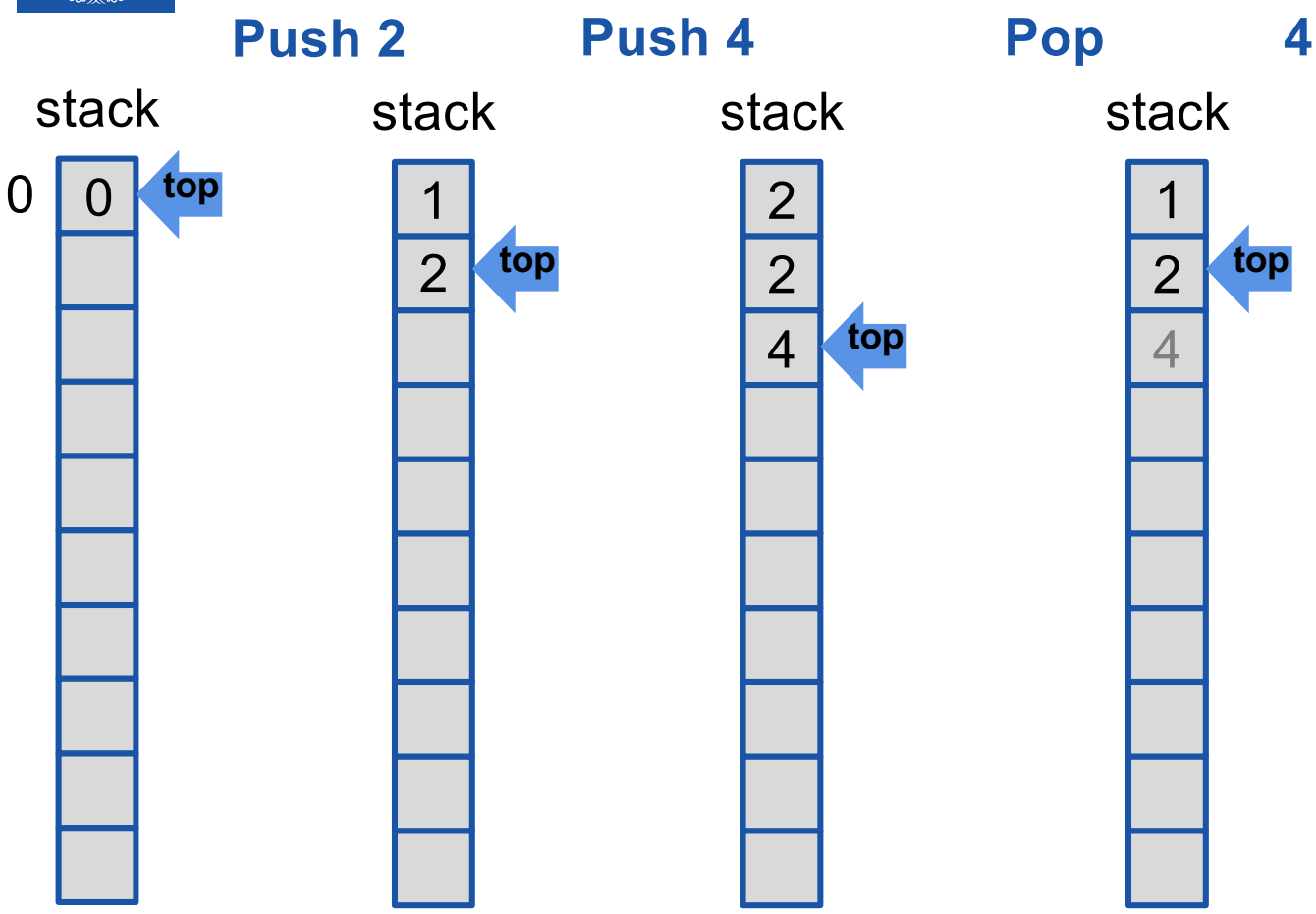
Gör ett minimalt main() av dina anteckningar! **2**

3 a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

(* Ni får inte göra det i Lab #2, ni ska träna på att använda parametrar.)

RPN miniräknare, Chapter10: P6



Algoritmer & Datastrukturer **0**

1 Skriv ner problemet så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! **2**

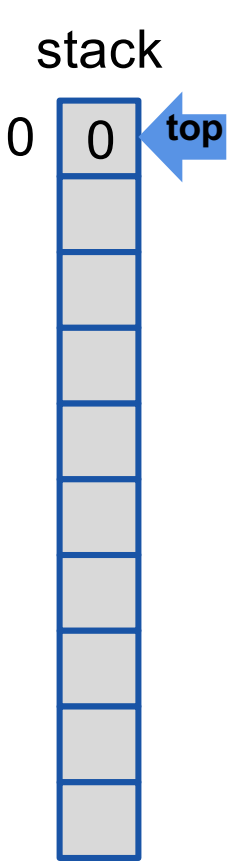
3 a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

SIZE

(Pop behöver inte ”radera” ett tal som tas bort, det kommer att skrivas över vid behov.)

RPN miniräknare, Chapter10: P6




```c
#include <stdio.h>
#define STACKSIZE 10

//initStack
//printStack
//pushStack
//popStack

int main(int argc, char **argv) {
    int stack[STACKSIZE];
    printf("RPN Calculator 0.1\n");
    //initStack
    //printStack
    //pushStack 4
    //printStack
    //pushStack 8
    //printStack
    //print popStack
    //printStack

    printf("RPNC Done, bye!\n");
    return 0;
}
```

```
RPN Calculator 0.1
RPNC Done, bye!
```

Algoritmer & Datastrukturer **0**

1 Skriv ner proble-met så att ett barn förstår det!

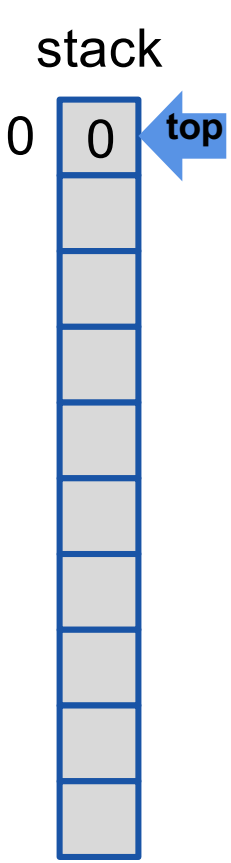
Gör ett minimalt main() av dina anteckningar! **2**

3 a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

Bra! Nu skulle vi behöva initStack & printStack…

RPN miniräknare, Chapter10: P6




```c
#include <stdio.h>
#de
void initStack(int aStack[]){
    aStack[0]=0;
}
void printStack(int aStack[]){
    printf("[ ");
    for (int i=aStack[0]; i>0; i--)
        printf("%d ", aStack[i]);
    printf("]\n");
}
//pushStack
//popStack

int main(int argc, char **argv) {
    int stack[STACKSIZE];
    printf("RPN Calculator 0.2\n");
    initStack(stack);
    printStack(stack);
    printf("RPNC Done, bye!\n");
    return 0;
}
```

(RPN räknaren behöver inte printStack() men ta för vana att alltid skapa en printfunktion till dina datastrukturer!)

```
RPN Calculator 0.2
[ ]
RPNC Done, bye!
```

Algoritmer & Datastrukturer **0**

1 Skriv ner proble-met så att ett barn förstår det!

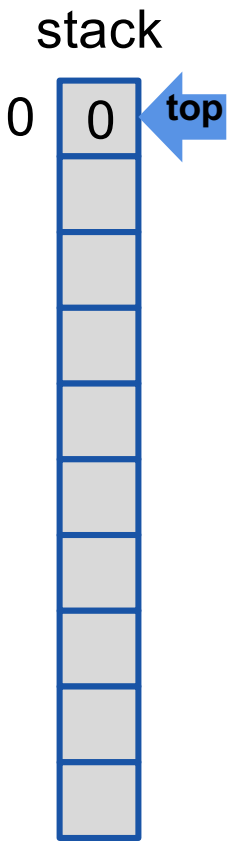
Gör ett minimalt main() av dina anteckningar! **2**

3 a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

Bra! Nu skulle vi behöva pushStack…

RPN miniräknare, Chapter10: P6




```c
#include <stdio.h>
#de
void initStack(int aStack[]){
    aStack[0]=0;
}
void printStack(int aStack[]){
    printf("[ ");
    for (int i=aStack[0]; i>0; i--)
int
}
//pushStack
//popStack

int main(int argc, char **argv) {
    int stack[STACKSIZE];
    printf("RPN Calculator 0.2\n");
    initStack(stack);
    printStack(stack);
    printf("RPNC Done, bye!\n");
    return 0;
}
```

…**push** ökar top med ett, och placerar elementet på den utpekade positionen…

Algoritmer & Datastrukturer **0**

1 Skriv ner problemet så att ett barn förstår det!

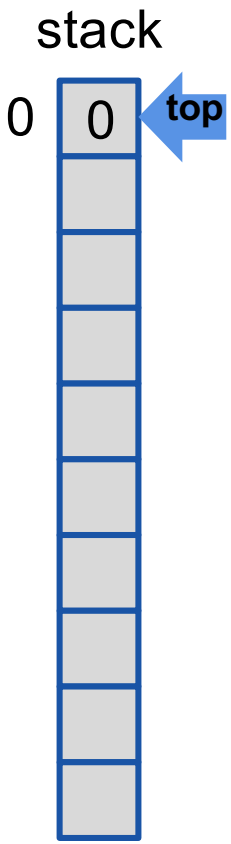
Gör ett minimalt main() av dina anteckningar! **2**

3 a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

Bra! Nu skulle vi behöva pushStack…

RPN miniräknare, Chapter10: P6




```c
#include <stdio.h>
void initStack(int aStack[]){
}
void pushStack(int aStack[], int aValue){
    aStack[++aStack[0]]=aValue;
}
//popStack

int main(int argc, char **argv) {
    int stack[STACKSIZE];
    printf("RPN Calculator 0.3\n");
    initStack(stack);
    printStack(stack);
    pushStack(stack, 4);
    printStack(stack);
    pushStack(stack, 8);
    printStack(stack);
    printStack(stack);
    printf("RPNC Done, bye!\n");
    return 0;
}
```

```
RPN Calculator 0.3
[ ]
[ 4 ]
[ 8 4 ]
RPNC Done, bye!
```

Algoritmer & Datastrukturer **0**

1 Skriv ner problemet så att ett barn förstår det!

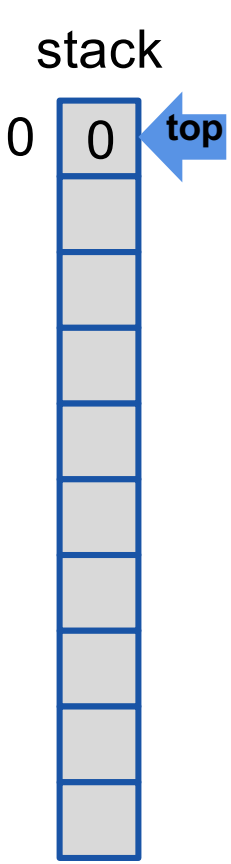
Gör ett minimalt main() av dina anteckningar! **2**

3 a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

Bra! Nu skulle vi behöva popStack…

RPN miniräknare, Chapter10: P6




```
#include <stdio.h>
#de
void initStack(int aStack[]){
    void pushStack(int aStack[], int aValue){
        aStack[++aStack[0]]=aValue;
    }
voi}
    //popStack

int
    int
}
//p
//p
    initStack(stack);
    printStack(stack);
int pushStack(stack, 4);
    printStack(stack);
    pushStack(stack, 8);
    printStack(stack);
    printStack(stack);
    printf("RPNC Done, bye!\n");
    return 0;
}
```

…**pop** returnerar sist lagrade element och räknar ner top med ett…

Algoritmer & Datastrukturer **0**

1 Skriv ner problemet så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! **2**

3 a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

Bra! Nu skulle vi behöva popStack…

RPN miniräknare, Chapter10: P6



Algoritmer & Datastrukturer **0**

1 Skriv ner problemet så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! **2**

3 a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

RPN miniräknare, Chapter10: P6

Backa bandet, nu har vi 4 bra rutiner…
…vad skulle vi egentligen göra?

Tänk dig att du har en fjädrande behållare som ”talrikshållaren i matsalen”. Tal ”sparas”, om det däremot är en operation så tas de två översta talen från behållaren och operationen utförs innan svaret sparas i behållaren! M.a.o

Repetera…
- Läs in nästa tecken…
- Om tal, tryck på stacken!
- Om op, utför op med de två översta talen på stacken!

…så länge tecknet är tal eller op!

Algoritmer & Datastrukturer **0**

1 **Skriv ner proble-met så att ett barn förstår det!**

Gör ett minimalt main() av dina anteckningar! **2**

3 **a..n** **Konstanter Variabler Uttryck & resultat!**

Läs in riktiga värden! **4**

Enbart ental, + & *, avsluta med lika med, t.ex: 1 2 3 * + = **(TIPS: M.a.o läs in med hjälp av %c!)**

RPN miniräknare, Chapter10: P6

```
int main(int argc, char **argv) {
    int stack[STACKSIZE]; char cmd;
    printf("RPN Calculator 1.0\n");
    initStack(stack);

    do {
        scanf(" %c", &cmd);
        switch (cmd){
            case '+' : pushStack(stack,
                                 popStack(stack)+popStack(stack));
                       break;
            case '*' : pushStack(stack,
                                 popStack(stack)*popStack(stack));
                       break;
            case '=' : popStack(stack); break;
            default  : pushStack(stack,cmd-'0');
        }
        printStack(stack);
    } while (cmd!='=');

    printf("RPNC Done, bye!\n");
    return 0;
}
```

```
RPN Calculator 1.0
1 2 3 * + =
[ 1 ]
[ 2 1 ]
[ 3 2 1 ]
[ 6 1 ]
[ 7 ]
[ ]
RPNC Done, bye!
```

Algoritmer & Datastrukturer **0**

1 Skriv ner problemet så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! **2**

3 a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

Bra applikation & en återanvändbar modul (Stack)!

RPN miniräknare, Chapter10: P6

```
int main(int argc, char **argv) {
    int stack[STACKSIZE]; char cmd;
    printf("RPN Calculator 1.0\n");
    initStack(stack);
```

Nästan helt perfekt, tre saker skulle kunna bli bättre:

1) Programmeraren vet att stacken är en matris
2) Programmeraren kan ändra i stack på egen hand!
3) Stacken kan bara hantera heltal!

Inte helt lätt att komma runt i c. OS-kursen visar på vägar runt problemet i c, och java-kursen visar hur ett objektorienterat språk lätt hanterar problemen.

```
    printf("RPNC Done, bye!\n");
    return 0;
}
```

Algoritmer & Datastrukturer **0**

1 **Skriv ner proble-met så att ett barn förstår det!**

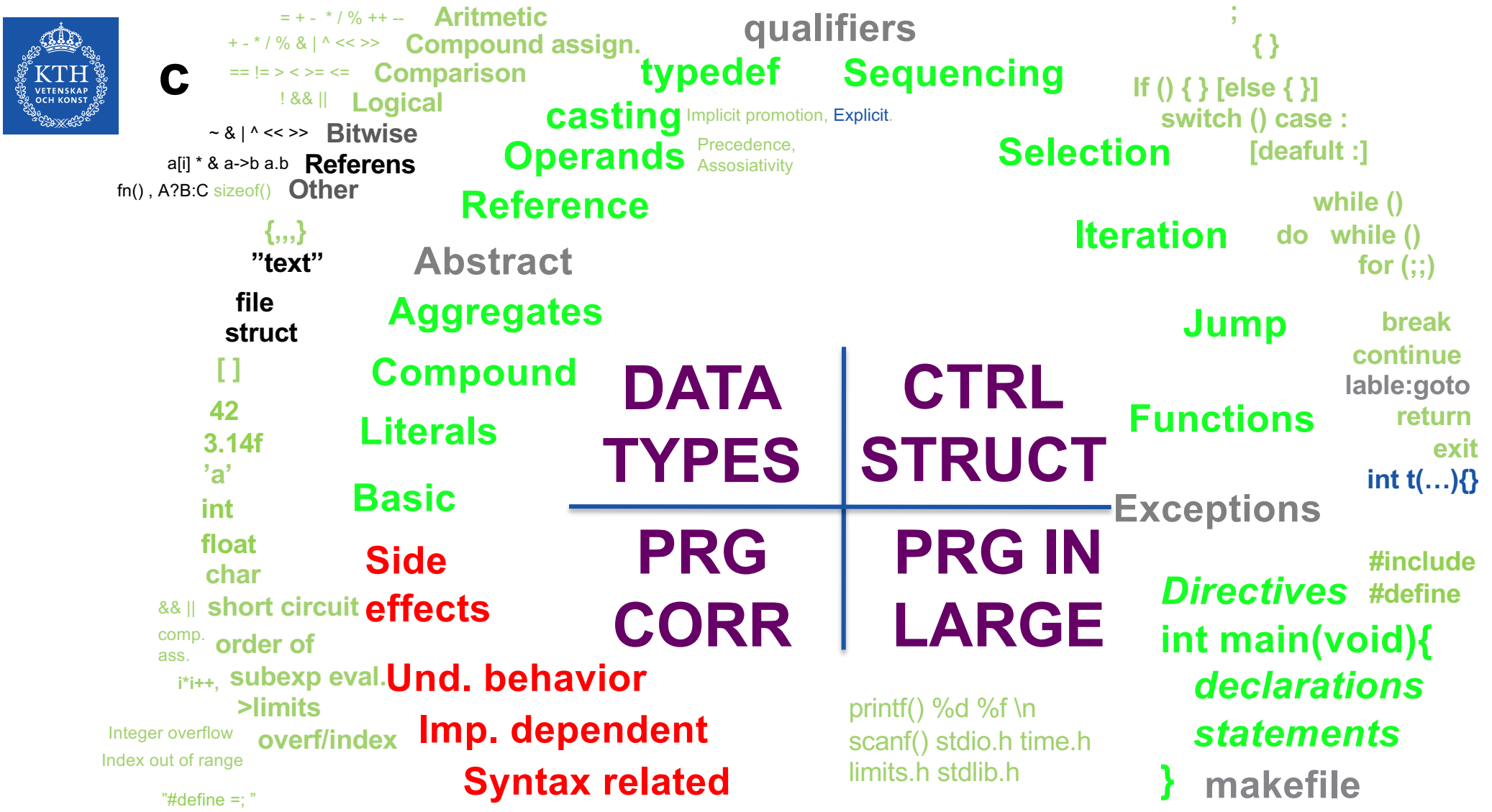
```
// creating stack
Stack st = new Stack();

// populating stack
st.push("Java");
st.push("Source");
st.push("code");
```

a..n resultat!

Läs in riktiga värden! **4**

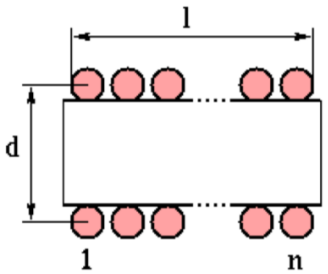
Att försöka gömma datastrukturen kallas för: Information hiding.


C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Compound
Literals
Abstract
Aggregates
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence,
Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
} makefile
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
>limits
Integer overflow
Index out of range
overf/index
Und. behavior
Imp. dependent
Syntax related
”#define =; ”
printf() %d %f \n
scanf() stdio.h time.h
limits.h stdlib.h
int/int
/*comment*/ or //

Övn #8: TEN2 Typuppgift #1

Du ska nog helst klara den på under 30 minuter!

1. (4p) Enkla luftlindade spolar är en viktig komponent radiosammanhang. Formeln för en sådan spoles induktans kan beräknas med följande formel:



$L = (d^2n^2)/(l+0.45d)$ [uH], där d och l anges i **meter**.

Du ska skriva ett enkelt program som frågar efter spolens **diameter i mm**, spolens **längd i mm** samt **hur många varv** tråd spolen består av. Du ska därefter med hjälp av formeln beräkna spolens induktans. Svaret ska presenteras med **2 decimaler** och uttryckt **i enheten uH**. TIPS: Tänk på att int / int blir en int!

Programmet ska också innehålla möjligheten att göra en ny beräkning! Ditt program bör likna följande exempel!

```
Single layer coil inductans calcylator.
Enter diameter in mm ?10
Enter length in mm ?10
Enter number of turns ?10
Inductans is 0.69 uH!
Another calculation [1=Yes, 0=No] ?1
Enter diameter in mm ?12
Enter length in mm ?12
Enter number of turns ?12
Inductans is 1.19 uH!
Another calculation [1=Yes, 0=No] ?0
SLCIC Done.
```



1 Skriv ner problemet så att ett barn förstår det!

2 Gör ett minimalt main() av dina anteckningar!

3 Konstanter
Variabler med Värde Uttryck
a..n & resultat!

4 Läs in riktiga värden!

Övn #8: TEN2 Typuppgift #2

Du ska nog helst klara den på under 60 minuter!

2. (4p) När Morse tänkt ut att bokstäver skulle kunna sändas som korta och långa signaler blev frågan vilken kod varje bokstav skulle få. Det sägs att han då tog en bok ur bokhyllan och noterade hur många gånger varje enskild bokstav fanns på en slumpmässigt vald sida. Han fann att bokstaven 'e' var vanligast följt av 't', 'i', 'a', 'n' och 'm'. Han gav därför de bokstäverna de kortaste koderna '.', '-', '..', '.-', '-.' & '--'. Nu är frågan: Hade Samuel Morse rätt?

För att undersöka det behövs med fördel en lång text men ditt program nöjer sig med att analysera en lång mening. Med andra ord: Uppgiften är att skapa ett program som läser in en lång mening **som ska lagras i en array**. Därefter ska programmet gå igenom arrayen och notera förekomsten av varje enskild bokstav i en frekvenstabell. Sist ska programmet skriva ut både frekvenstabellen och vilken bokstav som var vanligast. Skulle flera bokstäver få samma toppnotering räcker det om en av dessa presenteras. En körning bör se ut som följande exempel: (**Programmet ska enbart hantera 'A'..'Z'**.)

```
Text Letter Frequence Analyser.

Please enter a short text: SITTING IN AN ENGLISH GARDEN WAITING FOR THE SUN
IF THE SUN DONT COME YOU GET A TAN FROM STANDING IN THE ENGLISH RAIN
 A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
 7 0 1 3 8 3 7 5 11 0 0 2 2 15 5 0 0 4 6 10 3 0 1 0 1 0
Most frequent letter is 'N' found 15 times.
TLFA - Done.
```

1 Skriv ner problemet så att ett barn förstår det!

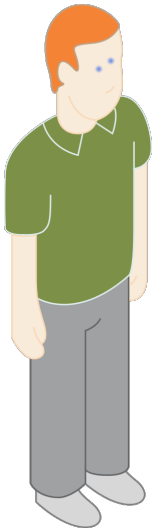
2 Gör ett minimalt main() av dina anteckningar!

3 Konstanter Variabler med Värde Uttryck & resultat!

a..n

4 Läs in riktiga värden!

Historik



- 150724 0.1 AC Rekursion.
- 150725 0.5 AC Scope
- 150726 1.0 AC Prg.utv.metodik.
- 160725 1.1 AC Mindre uppdateringar.
- 190801 2.0 AC Omarbetad från grunden, samt anpassad till KTH:s nya profil.